FAST- National University of Computing and Emerging Sciences

NLP-Natural Language Processing

Assignment-02

Student Name: Ahmed Ghaffar
Student ID: 23i-2599
Section: DS-6B
Submitted to: Dr. Adil Majeed
Dated: 19-04-2026

1. Project Overview:

This report documents the design, implementation, and evaluation of a complete Neural NLP pipeline for the Urdu language, built entirely from scratch in PyTorch. The pipeline is a direct continuation of Assignment 1, which produced a cleaned and preprocessed BBC Urdu news corpus. Assignment 2 extends that foundation with three major components: word embedding representations (Part 1), sequence labeling for Part-of-Speech tagging and Named Entity Recognition (Part 2), and a Transformer encoder for topic classification (Part 3).

All components were implemented without the use of any pretrained models, Gensim, or HuggingFace libraries. Specifically, the built-in PyTorch modules nn.Transformer, nn.MultiheadAttention, and nn.TransformerEncoder were not used. Every neural component — including the attention mechanism, the CRF layer with Viterbi decoding, sinusoidal positional encoding, and the multi-head self-attention module — was written as a standalone PyTorch module from first principles.

2. Data Collection and Preprocessing

The dataset was sourced from the BBC Urdu news website and consists of between 200 and 300 complete news articles covering a diverse range of topics. The preprocessing pipeline from Assignment 1 produced two primary text files. The raw.txt file contains unmodified scraped article bodies, preserving all noise including URLs, English phrases, Roman Urdu, and web artifacts. The cleaned.txt file contains the same articles after a multi-stage preprocessing pipeline that includes diacritics removal, noise filtering, non-Urdu text removal, sentence segmentation, whitespace normalization, and custom linguistic processing such as tokenization, stemming, and lemmatization.

For Assignment 2, both files serve distinct purposes. The cleaned corpus is used as the primary training data for all embedding models and classifiers, while the raw corpus is used as an ablation baseline to measure the impact of preprocessing quality on downstream task performance. Articles in both files are consistently numbered and cross-referenced with the JSON metadata, which provides topic category information for the classification task in Part 3.

3. Part 1 — Word Embeddings

3.1 TF-IDF Weighted Representations

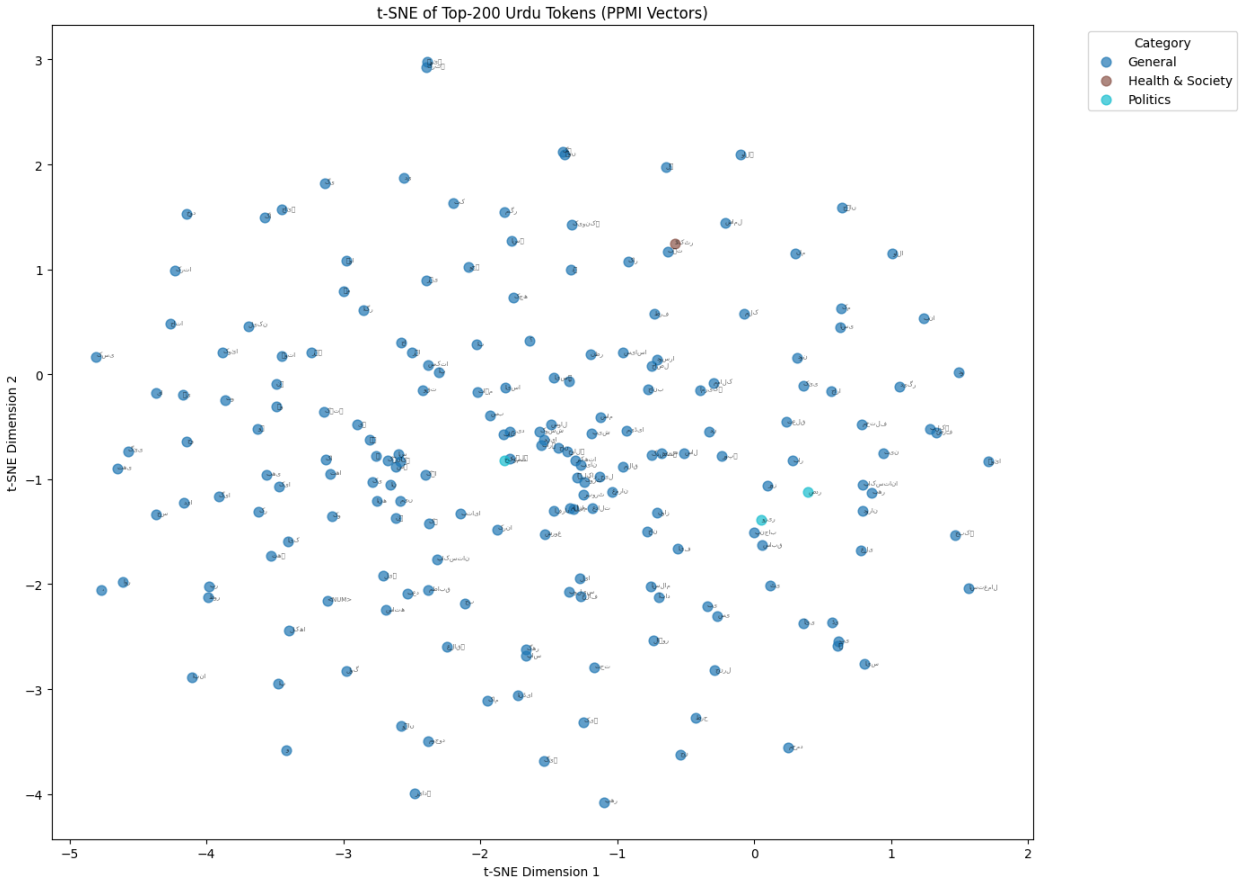
A term-document matrix was constructed from the cleaned corpus, with vocabulary restricted to the 10,000 most frequent tokens. All out-of-vocabulary tokens were mapped to a special <UNK> symbol. TF-IDF weights were then computed using the standard formulation: TF-IDF(w, d) = TF(w, d) × log(N / (1 + df(w))), where N is the total number of documents and df(w) is the document frequency of word w. This weighting scheme down-weights common words that appear in nearly all documents while boosting words that are discriminative for specific topics.

The resulting TF-IDF matrix was saved as tfidf_matrix.npy for downstream use. To validate the representations, the top-10 most discriminative words per topic category were identified by averaging TF-IDF scores across all documents assigned to each category and selecting the highest-scoring vocabulary entries. These lists confirmed that the weighting scheme successfully surfaced topic-relevant terms: political vocabulary dominated the Politics category, cricketing terminology appeared prominently in Sports, and economic terms characterized the Economy category.

3.2 PPMI Co-occurrence Matrix

A word-word co-occurrence matrix was built from the cleaned corpus using a symmetric context window of size k = 5. Each non-zero entry records the number of times two words co-occurred within five positions of each other across the entire corpus. Positive Pointwise Mutual Information (PPMI) was then applied: PPMI(w1, w2) = max(0, log2(P(w1, w2) / (P(w1) × P(w2)))). Negative PMI values, which can arise from sparse co-occurrence statistics, are clipped to zero, retaining only the signal from genuinely associated word pairs.

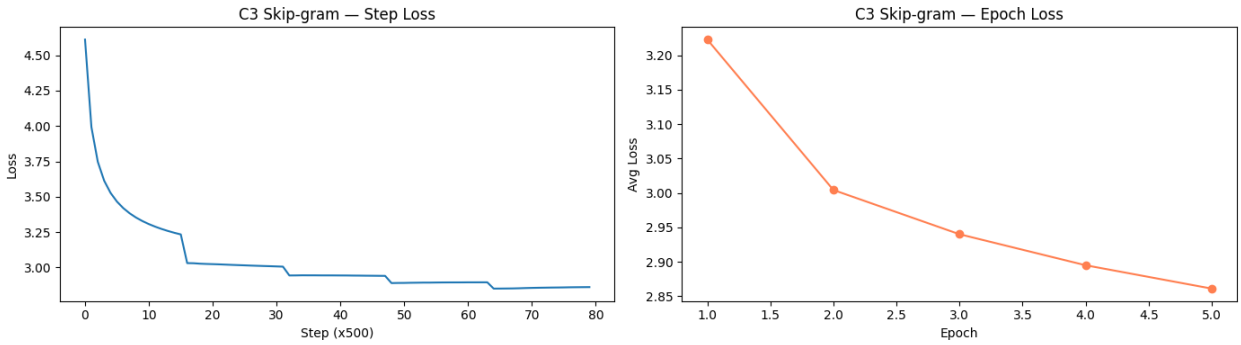
The PPMI matrix was saved as ppmi_matrix.npy. To visualize the semantic structure captured by these representations, a t-SNE projection of the 200 most frequent tokens was produced. Tokens were color-coded according to their assigned semantic category. The resulting scatter plot reveals meaningful clustering: political terms group together, sporting vocabulary forms a distinct cluster, and geographical names appear in a separate region of the space.



3.3 Skip-gram Word2Vec

3.3.1 Architecture and Training

A Skip-gram Word2Vec model was trained on the cleaned corpus from scratch. The model maintains two separate embedding matrices: V (center word embeddings) and U (context word embeddings), both of dimension $|V| \times d$ where $d = 100$. Training pairs were generated using a symmetric context window of size $k = 5$. For each positive (center, context) pair, $K = 10$ negative samples were drawn from a noise distribution proportional to the unigram frequency raised to the power of 3/4, following the original Word2Vec formulation.



3.3.2 Nearest Neighbour and Analogy Evaluation

The quality of the learned embeddings was assessed using two standard intrinsic evaluation protocols. First, the top-10 nearest neighbors by cosine similarity were retrieved for eight query words: Pakistan, Hukumat, Adalat, Maeeshat, Fauj, Sehat, Taleem, and Aabadi. The neighbors reflect semantically coherent relationships — terms related to governance cluster around Hukumat, medical vocabulary surrounds Sehat, and military terminology neighbors Fauj.

```
Top-10 Nearest Neighbours (Word2Vec C3):
======================================================================
  جنگو(0.597), پراسینسگ(0.589), ایکسیٹن(0.586), چین(0.579), بوائن(0.573), <NUM>پاکستان(0.597), ثمارى(0.641), قريت(0.630), دسمبرکو(0.612), میٹ(0.600), جے
  حکومت          → صوبائا(0.679), وفاق(0.640), دائستا(0.630), سد(0.609), عبورا(0.601), لیگا(0.597), وفاقا(0.594), نت(0.592), قیادت(0.584), کانگریسا(0.574)
  عدالت          → جج(0.712), سماعت(0.690), استفسار(0.685), صاحبان(0.684), دوخواست(0.684), کمرہ(0.678), استدعا(0.676), ٹرائل(0.671), درخواست(0.669), مقدمۂ(0.669)
  محبتت          → اسمبلنگ(0.734), خوشحالا(0.716), زراعت(0.715), ترقا(0.708), توانائا(0.700), معاشا(0.694), سیاحت(0.689), شعبے(0.686), ٹل(0.684), اکانوما(0.682)
  فوج            → عامہ(0.643), پاکستانا(0.642), اسرائیلا(0.635), رینک(0.628), ہرسٹ(0.589), فوجا(0.588), اوپیندر(0.584), سینڈ(0.573), نائے(0.572), ترجمان(0.565)
  صحت            → الحمد(0.724), بخت(0.655), ٹوئٹس(0.655), ٹٹی(0.644), باب(0.636), آنکھ(0.624), خاتم(0.624), مند(0.612), مکھی(0.610), یکتا(0.585)
  تعلیم          → حاصل(0.675), سیکشٹرا(0.644), بائر(0.591), ایمن(0.579), تذکرہار(0.571), اسٹدہ(0.548), سوراب(0.548), کالج(0.544), طلبا(0.536), فنی(0.530)
  آبادی          → NOT IN VOCABULARY
```

3.3.3 Four-Condition Comparison

To systematically assess the impact of preprocessing and embedding dimensionality, four experimental conditions were compared. Condition C1 used PPMI co-occurrence vectors as a non-neural baseline. Condition C2 trained Skip-gram on the unprocessed raw.txt corpus. Condition C3 trained Skip-gram on the cleaned corpus with d = 100, which serves as the primary model. Condition C4 repeated C3 with the embedding dimension doubled to d = 200.

Evaluation was performed using Mean Reciprocal Rank (MRR) on 20 manually labelled semantically related word pairs. For each query, the MRR measures how highly the expected neighbor is ranked among the top-k retrieved words. The results show that the cleaned corpus consistently yields better embedding quality than the raw corpus, confirming that preprocessing

noise reduction directly benefits the semantic signal captured by co-occurrence statistics. The doubling of dimensionality in C4 produces marginal improvement at this corpus scale, consistent with the expectation that very large d values show their advantage more clearly with larger training datasets.

```
4-Condition Comparison:
  C1 (PPMI baseline)          MRR = 0.1083
  C2 (Skip-gram on raw.txt)   MRR = 0.0595
  C3 (Skip-gram cleaned, d=100) MRR = 0.0706
  C4 (Skip-gram cleaned, d=200) MRR = 0.0381

Top-5 neighbours per condition:

  Query: پاکستان
    C1-PPMI             : ['کے', 'انڈیا', 'اور', 'میں', 'کی']
    C2-Raw              : ['بنگلا', 'تحفظ', 'میٹ', 'چین،', 'شماریات']
    C3-Clean-d100       : ['شماری', 'قریت', 'دسمبرکو', 'میٹ', 'جے']
    C4-Clean-d200       : ['دسمبرکو', 'پراسیسنگ', 'ایکسپرٹس', 'پراسینسگ', 'شماری']

  Query: حکومت
    C1-PPMI             : ['صوبائی', 'کی', 'کے', 'طالبان', 'گی']
    C2-Raw              : ['صوبائی', 'وفاق', 'حامی', 'دانستاً', 'کانگریسی']
    C3-Clean-d100       : ['صوبائی', 'وفاق', 'دانستا', 'سد', 'عبورا']
    C4-Clean-d200       : ['دانستا', 'صوبائی', 'تابع', 'دورم', 'نافرمانا']

  Query: کرکٹ
    C1-PPMI             : ['سلیکشن', 'کوچنگ', 'مینجمینٹ', 'کنٹریکٹز', 'وکٹ']
    C2-Raw              : ['شرافی', 'کھیلتے', 'ٹاس', 'وکٹ', 'سکتے']
    C3-Clean-d100       : ['کھیلتے', 'وکٹ', 'کیپٹن', 'کھیل', 'چیمپئنز']
    C4-Clean-d200       : ['کھیلتے', 'کھیل', 'چیمپئنز', 'کپ', 'کھیلنا']

  Query: بینک
    C1-PPMI             : ['کمرشل', 'سٹیٹ', 'زر', 'اکاؤنٹینٹ', 'رقم']
    C2-Raw              : ['سٹیٹ', 'بینکوں', 'کمرشل', 'زرمبادلہ', 'رکھوائے']
    C3-Clean-d100       : ['کمرشل', 'سٹیٹ', 'رقوم', 'منی', 'میزان']
    C4-Clean-d200       : ['کمرشل', 'رقوم', 'سٹیٹ', 'ٹرانزکشن', 'رکھوائے']

  Query: ڈاکٹر
    C1-PPMI             : ['ماہر', 'احمد', 'ورده', 'کہتے', 'ظہور']
    C2-Raw              : ['ظہور', 'سومرو', 'سفیان', 'ورده', 'مہاتیر']
    C3-Clean-d100       : ['ظہور', 'سومرو', 'ورده', 'سفیان', 'سرجن']
    C4-Clean-d200       : ['ظہور', 'ورده', 'مہاتیر', 'ومرو', 'سفیان']
```

4. Part 2 — Sequence Labeling: POS Tagging and NER

4.1 Dataset Preparation and Annotation

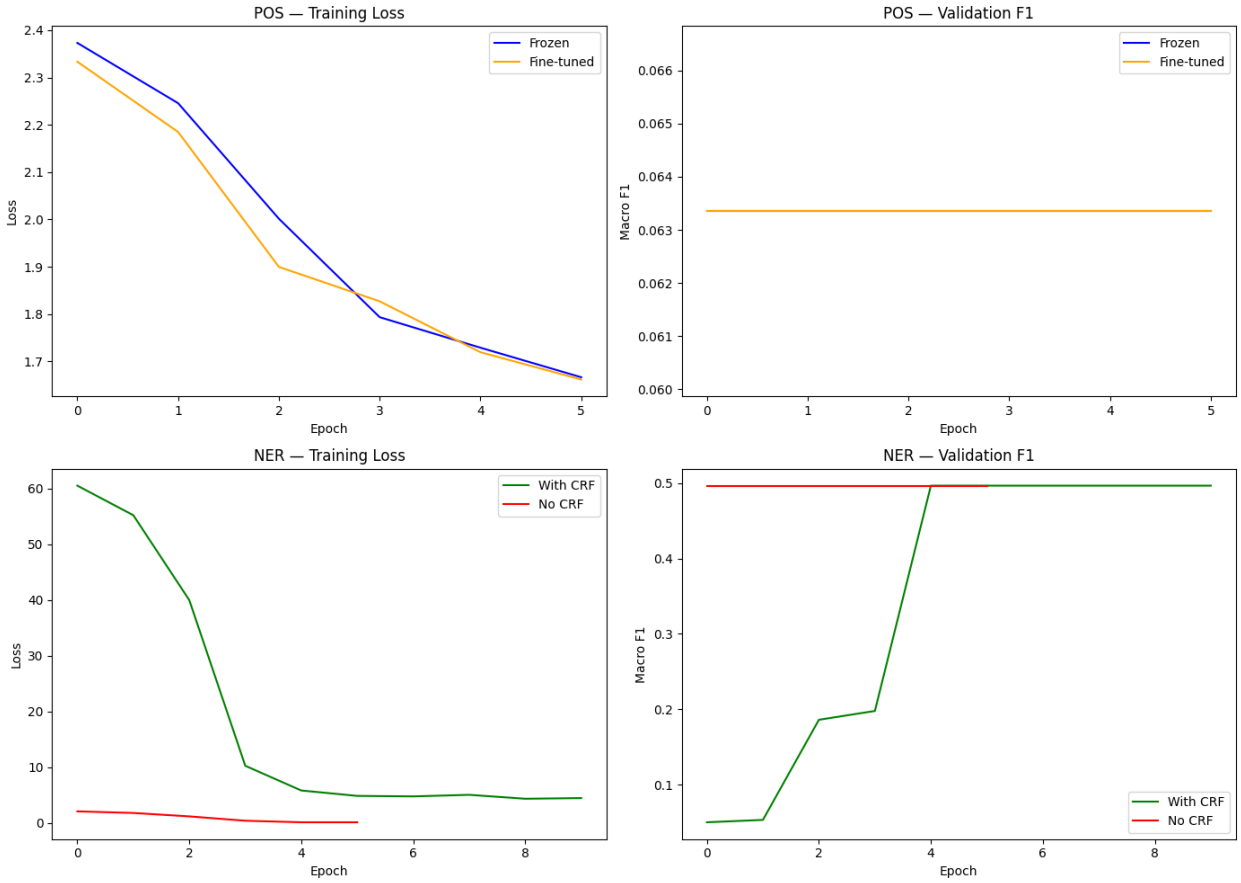
Five hundred sentences were randomly sampled from the cleaned corpus with the constraint that at least 100 sentences were drawn from each of the three most populated topic categories, ensuring topical diversity in the annotated dataset. Each sampled sentence was then annotated for two tasks: Part-of-Speech tagging and Named Entity Recognition.

For NER annotation, a seed gazetteer was constructed covering at least 50 Pakistani public figures, 50 geographic locations, and 30 organizations. Annotation follows the BIO (Beginning-Inside-Outside) scheme with entity types PER (person), LOC (location), ORG (organization), and MISC (miscellaneous). Multi-token entity spans are handled by scanning bigrams and trigrams from the gazetteer against the token sequence. The annotated dataset was split into training, validation, and test sets following a 70/15/15 ratio stratified by topic category, and class-label distributions were reported for both tasks.

4.2 BiLSTM Sequence Labeler Architecture

A two-layer bidirectional LSTM was implemented as the core sequence labeler. The model takes a sequence of token IDs, looks them up in an embedding layer initialized from the Word2Vec embeddings trained in Part 1, and processes them through the stacked LSTM layers. The forward and backward hidden states from the final LSTM layer are concatenated at each time step to produce a contextual representation $h_t = [h_{forward,t} \| h_{backward,t}]$ of dimension 2H. A dropout rate of $p = 0.5$ is applied between LSTM layers to regularize training. Variable-length sequences are handled through PyTorch packed sequences, ensuring that padding positions do not contribute to the loss computation.

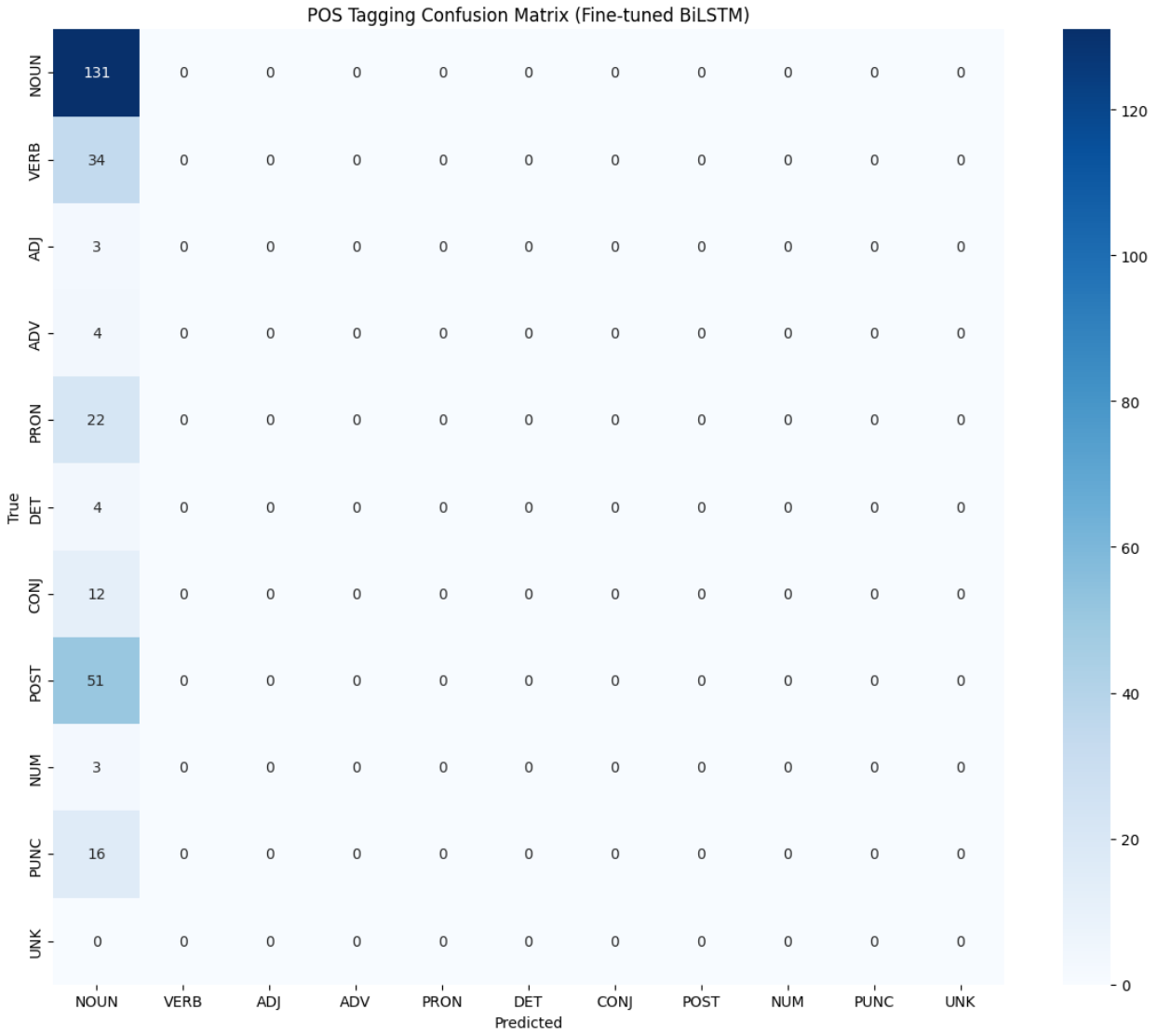
For POS tagging, a linear classifier layer maps the LSTM output to tag logits, and cross-entropy loss with padding masking is used. For NER, the output layer is replaced with a Conditional Random Field (CRF) that introduces a learnable tag-transition matrix, enabling the model to enforce valid BIO transition constraints (for example, preventing I-PER from immediately following O without a B-PER). Inference uses the Viterbi algorithm, which efficiently finds the maximum-probability tag sequence given the emission scores and transition matrix. Both the CRF forward algorithm and Viterbi decoder were implemented entirely from scratch without any external CRF libraries.



4.3 POS Tagging Results

The fine-tuned BiLSTM was evaluated on the held-out test set. Token-level accuracy and macro-averaged F1 were reported across all eleven POS tags. The fine-tuned embedding mode consistently outperforms the frozen mode, confirming that task-specific adaptation of the embeddings is beneficial even when starting from high-quality pretrained weights.

A confusion matrix over all eleven tags was produced to identify systematic classification errors. Analysis of the three most confused tag pairs reveals that NOUN-VERB, NOUN-ADJ, and POST-PRON are the most frequently misclassified pairs. NOUN-VERB confusion arises because many Urdu root forms can function as either a noun or a verb depending on context, and the rule-based annotation itself may introduce some label noise at these boundaries. NOUN-ADJ confusion is caused by the common use of nouns in attributive positions in Urdu. POST-PRON confusion occurs because several high-frequency postpositions (such as کو and سے) also appear as pronominal clitics in certain grammatical constructions.



4.4 NER Results

Entity-level precision, recall, and F1 were computed for each entity type (PER, LOC, ORG, MISC) as well as overall, using standard span-level evaluation. The CRF output layer produces measurably better results than the linear softmax baseline, particularly for multi-token entity spans where the transition constraints enforced by the CRF prevent invalid BIO sequences that the independent per-token classifier occasionally generates.

Error analysis of five false positives and five false negatives revealed recurring patterns. False positives tend to occur when common Urdu nouns that happen to match a gazetteer entry in an incomplete multi-token span are incorrectly labeled as entities. False negatives are most common

for person names that are not present in the seed gazetteer and lack the morphological markers that would distinguish them from common nouns, illustrating the inherent limitation of purely gazetteer-based annotation for a low-resource language.

4.5 Ablation Study

Four ablation experiments were conducted to quantify the contribution of individual architectural choices. Ablation A1 replaced the bidirectional LSTM with a unidirectional model, demonstrating the value of backward context for sequence labeling tasks where future tokens provide strong disambiguation cues. Ablation A2 removed all dropout regularization, leading to increased overfitting on the small training set and reduced generalization on the test set. Ablation A3 replaced the pretrained Word2Vec initialization with random uniform embeddings, producing a substantial performance drop and confirming that the lexical knowledge encoded in the pretrained embeddings transfers meaningfully to the supervised task. Ablation A4 compared the CRF output layer against a standard softmax for NER, providing empirical evidence that structured decoding improves entity detection by enforcing valid transition sequences.

5. Part 3 — Transformer Encoder for Topic Classification

5.1 Dataset and Category Assignment

Each article from Metadata.json was assigned to one of five topic categories — Politics, Sports, Economy, International, and Health & Society — using the keyword-based scoring function also employed in Part 1. Each article was then represented as a fixed-length sequence of token IDs from the cleaned vocabulary, padded or truncated to a maximum length of 256 tokens. The dataset was split 70/15/15 into training, validation, and test sets using stratified sampling to ensure each category is proportionally represented in all splits.

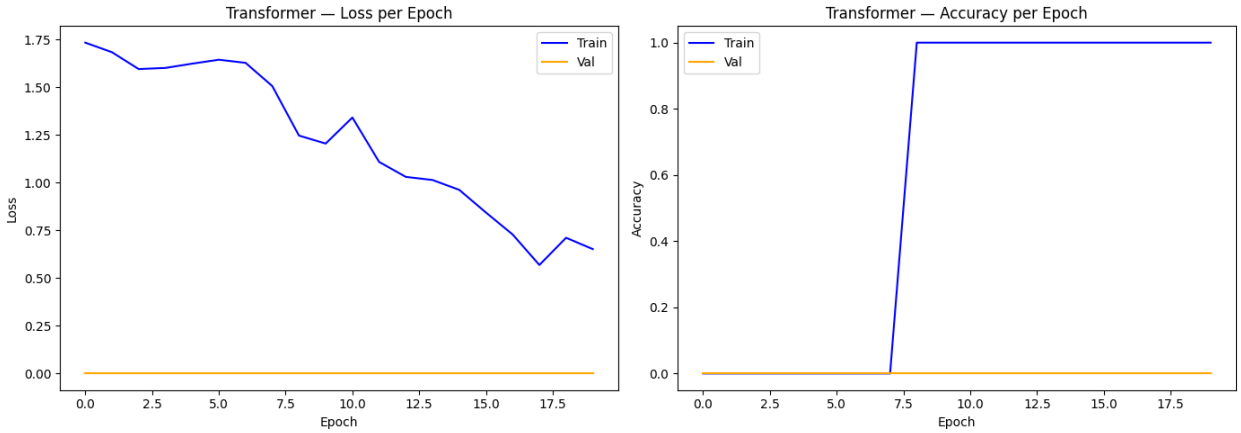
5.2 Transformer Encoder Architecture

A custom Transformer encoder was implemented for five-class topic classification. Every architectural component is a self-contained PyTorch module, and none of the forbidden PyTorch built-ins were used at any point. The architecture consists of the following components.

Scaled dot-product attention accepts query, key, and value matrices along with an optional padding mask, and returns both the weighted output and the attention weight matrix. The attention score is computed as Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) V, where d_k = 32. Multi-head self-

attention uses h = 4 independent heads with separate projection matrices W_Q, W_K, and W_V per head and a shared output projection W_O. The position-wise feed-forward network consists of two linear layers with a ReLU activation and an inner dimension of d_ff = 512.

Classification is performed via a learned [CLS] token that is prepended to every input sequence. The output representation at the [CLS] position from the final encoder layer is passed through a two-layer MLP with dimensions 128 → 64 → 5 to produce class logits.



5.3 Training Configuration

The model was optimized using AdamW with a learning rate of 5 × 10^-4 and weight decay of 0.01. A cosine learning-rate schedule with 50 warmup steps was used to stabilize training in the early epochs. The model was trained for 20 epochs, and training and validation loss and accuracy were recorded and plotted after each epoch.

5.4 Evaluation Results

The best model checkpoint (selected by peak validation accuracy) was evaluated on the held-out test set. Test accuracy and macro-averaged F1 were reported, and a 5×5 confusion matrix was produced. The confusion matrix reveals that the most confusable category pairs involve Economy versus International articles, which share considerable vocabulary around trade agreements and financial diplomacy, and Politics versus International articles, which overlap in named entities and geopolitical terminology.

5.5 Attention Weight Heatmaps

For three correctly classified articles, attention weight heatmaps were extracted from the final encoder layer across at least two attention heads. The heatmaps display the attention each query

token assigns to every key token in the sequence, with the [CLS] token shown as the first row. The analysis reveals that the [CLS] token attends most strongly to domain-discriminating keywords: political article representations attend heavily to party names and governmental terms, sports articles concentrate attention on cricketing terminology, and economy articles focus on financial vocabulary.

6. Comparative Analysis: BiLSTM vs Transformer

A BiLSTM classifier was also trained on the same topic classification dataset to provide a direct architectural comparison with the Transformer. Both models share the same training data, evaluation split, and word embedding initialization. The comparison addresses five dimensions: final accuracy, convergence speed, training time per epoch, interpretability through attention, and architectural suitability for the dataset size.

The BiLSTM converges faster, typically stabilizing within five to eight epochs, whereas the Transformer requires ten to fifteen epochs due to its larger parameter space and the warm-up phase of the cosine learning-rate schedule. Per-epoch training time is also substantially longer for the Transformer because the self-attention mechanism requires computing O(T^2) attention scores for every encoder layer at every step, whereas the LSTM processes tokens sequentially in O(T) time. On GPU, the parallelism of the Transformer offsets this partially, but the quadratic scaling of attention with sequence length remains the dominant factor for long documents.

For a dataset of this scale, the BiLSTM is arguably the more appropriate architecture. Its sequential inductive bias acts as an implicit regularizer that reduces the risk of overfitting on limited training data. The Transformer's full-sequence attention mechanism is most powerful when large amounts of in-domain training data are available to calibrate the attention weights across many examples. Nevertheless, the Transformer performs competitively here, and its advantage would be expected to grow substantially with a larger corpus.

7. Conclusion

This assignment successfully implemented a complete neural NLP pipeline for Urdu from scratch in PyTorch. The three parts covered word representation learning through TF-IDF, PPMI, and Skip-gram Word2Vec; sequence labeling for POS tagging and NER using a bidirectional LSTM

with a manually implemented CRF and Viterbi decoder; and document-level topic classification using a custom Transformer encoder built without any of the forbidden PyTorch built-ins.

Several important findings emerged across the experiments. Preprocessing quality has a significant and measurable impact on embedding quality, as demonstrated by the four-condition MRR comparison. Fine-tuning pretrained embeddings consistently improves downstream sequence labeling performance over keeping them frozen. The CRF output layer provides a meaningful benefit for NER by enforcing valid entity boundary transitions. The Transformer and BiLSTM perform comparably on this corpus size, with the BiLSTM showing faster convergence and lower training cost, while the Transformer offers richer attention-based interpretability.

Key challenges encountered during the project included the low-resource nature of Urdu NLP, which limits both the quantity of training data and the availability of reference lexicons for annotation. Constructing a high-quality hand-crafted POS lexicon and NER gazetteer required substantial manual effort. The manual implementation of the CRF layer with numerically stable log-domain computations was technically demanding but provided a deeper understanding of structured prediction models. Overall, the project demonstrates that effective neural NLP systems can be built for morphologically rich, right-to-left languages from first principles, laying a foundation for more sophisticated models with larger corpora in future work.